

# EIOS INFRASTRUCTURE ANALYSIS

## Đánh giá: Kiến trúc Ollama Fleet của bạn vs Eios Enterprise Requirements

---

### EXECUTIVE SUMMARY

Kiến trúc hiện tại của bạn (Ollama Fleet Router + SQLite):

-  **Tuyệt vời cho:** Phase 1 MVP, Development, Small-scale self-hosted
  -  **Không đủ cho:** Phase 2+ Enterprise, Hàng ngàn agent đồng thời
  -  **Cần bổ sung:** Orchestration engine, Message broker, Vector DB, API Gateway
- 

## 1 PHÂN TÍCH ĐIỂM CÓ & ĐIỂM THIẾU

### ĐIỂM CÓ - Kiến trúc hiện tại

#### 1.1 Load Balancing thông minh (ollama-fleet-router)

Ưu điểm:

- Điều phối request tới 3 Ollama instances
- Có thể pre-warm models (giữ models trong VRAM)
- Giảm latency so với khởi động mô hình lạnh
- Cho phép round-robin hoặc weighted routing

Sử dụng cho Agent:

backend-coder → routed to Ollama 01 (qwen3.6) ← Best for code generation  
qa-tester → routed to Ollama 02 (gemma4) ← Best for test writing  
data-analyst → routed to Ollama 03 (llama3) ← Best for SQL queries

#### 1.2 SQLite Router Database

Ưu điểm:

- Single binary, zero deployment overhead
- Đủ cho development + small team (< 10 concurrent executions)
- Tốc độ nhanh với SELECT/INSERT đơn giản
- Dễ backup (copy file .db)

Schema tối giản:

```

executions: id, agent_id, status, created_at, tokens_used
execution_logs: execution_id, timestamp, log_text
models: model_name, endpoint, status

```

## 1.3 Monitoring Dashboard

Ưu điểm:

- Realtime theo dõi health của Ollama fleet
- Track model inference latency, error rates
- Simple metrics (uptime, requests/sec)

## ĐIỂM THIẾU - Các yêu cầu Eios không được đáp ứng

### 1. Không có Workflow Orchestration Engine

**Vấn đề:**

Eios yêu cầu State Machine phức tạp:

```

QUEUED → PLANNING → EXECUTING → AWAITING_REVIEW → COMPLETED/FAILED

```

Nếu dùng SQLite:

- Không handle long-running workflows (> 30 phút)
- Không có automatic retry với exponential backoff
- Không thể suspend/resume execution (nếu AI cần review)
- Không handle timeout (timeout 24h cho review)

## Giải pháp: Cần Temporal.io hoặc Cadence

```

// Temporal Workflow example
@Workflow
class AgentExecutionWorkflow {
    @WorkflowMethod
    async execute(execution_id: string) {
        try {
            const plan = await planner.createPlan();           // PLANNING state

```

```

    const action = await executor.run(plan);          // EXECUTING state

    if (action.needs_review) {
        const decision = await activities.waitForReview(30 * 60); // 30min timeout
        if (decision === 'REJECTED') {
            return await this.execute(execution_id); // RE-PLAN
        }
    }

    return { status: 'COMPLETED' };
} catch (e) {
    return { status: 'FAILED', error: e };
}
}
}

```

## ⚠️ 2. SQLite không scale cho Multi-team/Multi-agent

### Vấn đề:

Giới hạn SQLite:

- Single writer lock (chỉ 1 write transaction tại một lúc)
- Nếu 100 agent chạy đồng thời + mỗi cái ghi 1 execution\_step/sec
  - Database lock → Latency spike → Timeout

Ví dụ:

$10 \text{ agents} \times 60 \text{ exec/day} \times 5 \text{ steps/exec} = 3000 \text{ writes/day (OK)}$   
 $1000 \text{ agents} \times 60 \text{ exec/day} \times 5 \text{ steps/exec} = 300K \text{ writes/day (FAIL)}$

### Giải pháp: Chuyển sang PostgreSQL + Redis

PostgreSQL:

- Multi-writer, handles 1000+ concurrent connections
- JSONB columns cho flexibility (config, metadata)
- Triggerfunction để update updated\_at tự động
- WAL (Write-Ahead Log) cho durability

Redis:

- Cache hot data (agent config, active executions)
- pub/sub cho real-time notifications (WebSocket)
- Queue (executions waiting to be processed)

## ⚠️ 3. Không có Message Broker cho Multi-agent Orchestration

## Vấn đề:

Eios hỗ trợ Subagents (1 agent gọi agent khác). Ví dụ:

```
backend-coder (parent) → code review → qa-tester (child)
```

Nếu dùng SQLite + direct HTTP calls:

- Nếu qa-tester bị timeout, parent mình tạm dừng (không handle gracefully)
- Không có priority queue (HIGH risk actions nên được ưu tiên)
- Không có deadletter queue cho failed messages

## Giải pháp: Kafka hoặc Redis Streams

Topics (Kafka):

```
execution-started: { execution_id, agent_id, timestamp }
execution-step-completed: { step_id, result, tokens_used }
review-requested: { review_id, risk_level, action }
execution-completed: { execution_id, status, total_cost }
```

Subagent call flow:

1. backend-coder parent\_execution posts "task\_for\_qa" → Kafka topic
2. qa-tester consumer polls topic → creates child\_execution
3. qa-tester completes → posts "task\_result" → parent listener
4. parent continues → EXECUTING → COMPLETED

## ⚠ 4. Không có Vector Database (Long-term Memory)

### Vấn đề:

Eios Agent có bộ nhớ dài hạn (giống Claude's memory). Ví dụ:

- Agent sửa lỗi mỗi tuần → lưu context từ những lần sửa trước
- Khi user hỏi "Fix similar to the bug from last month", agent cần tìm lại execution cũ có context tương tự

SQLite không phù hợp:

- Không hỗ trợ semantic search (tìm text tương tự)
- Phải load toàn bộ execution\_logs lên memory → slow

## Giải pháp: Qdrant (Vector DB)

```
# RAG Workflow
embeddings = embed_text("Fix the memory leak") # ada-002 embedding
similar = qdrant.search(
    vector=embeddings,
    limit=3,
```

```
    filter={"agent_id": "backend-coder"}
)
# Returns: 3 previous executions có context tương tự
# Inject vào LLM context: "Based on similar issues..."
```

## ⚠ 5. Không có API Gateway (Rate Limiting, Zero Trust)

### Vấn đề:

Eios yêu cầu:

- Rate limit per team (Team A: 100 exec/day, Team B: 1000 exec/day)
- Zero Trust: Mỗi request phải verify JWT token + team ownership
- DDoS protection (nếu hacker spam /executions endpoint)
- Request tracing (correlate log từ multiple services)

Nếu direct expose Ollama router:

- Ai cũng có thể call /v1/models → leak thông tin
- Ai cũng có thể call model mà không cần authenticate
- Không track token usage per API key

### Giải pháp: Kong hoặc APISIX

```
# Kong Config
routes:
- name: "eios-execution"
  paths: ["/api/v1/agents/{id}/executions"]
  upstream: temporal-api-service
  plugins:
    - name: jwt
      config:
        secret: ${JWT_SECRET}
    - name: rate-limiting
      config:
        minute: 100 # 100 requests/minute
    - name: request-transformer
      config:
        add:
          headers:
            X-Team-ID: ${jwt.team_id} # Inject team từ JWT

- name: "eios-llm"
  paths: ["/api/v1/llm"]
  upstream: llm-gateway-service
  plugins:
    - name: request-size-limiting
```

```
config:
  size: 10 # 10MB max
```

## ⚠ 6. MCP Servers không có isolated/sandboxed execution

### Vấn đề:

```
eios.yaml define:
  mcp_servers:
    - postgres-db (read_only: true trên users, orders tables)
    - company-wiki (SSE)
```

Nếu SQLite + simple router:

- Không có validation layer giữa Agent → MCP
- Nếu AI bị prompt injection, có thể bypass read\_only  
Example: AI outputs "ALTER TABLE users..." → MCP execute thẳng
- Không có audit log cho tất cả MCP calls

### Giải pháp: MCP Validation Gateway

```
class MCPValidationLayer {
  async validateCall(
    toolName: string,
    args: unknown,
    agentConfig: AgentConfig
  ) {
    // 1. Input schema validation
    validateInputSchema(toolName, args);

    // 2. Deny-list checking (from eios.yaml)
    const deniedPatterns = agentConfig.permissions[toolName]?.deny_list || [];
    for (const pattern of deniedPatterns) {
      if (args.command?.match(new RegExp(pattern))) {
        throw new PermissionError('Command denied by policy');
      }
    }

    // 3. SQL mutation check (dù AI cố gắng đánh lừa)
    if (toolName === 'query_database' && !args.sql.trim().toUpperCase().startsWith('SELECT ')) {
      throw new PermissionError('Only SELECT queries allowed');
    }

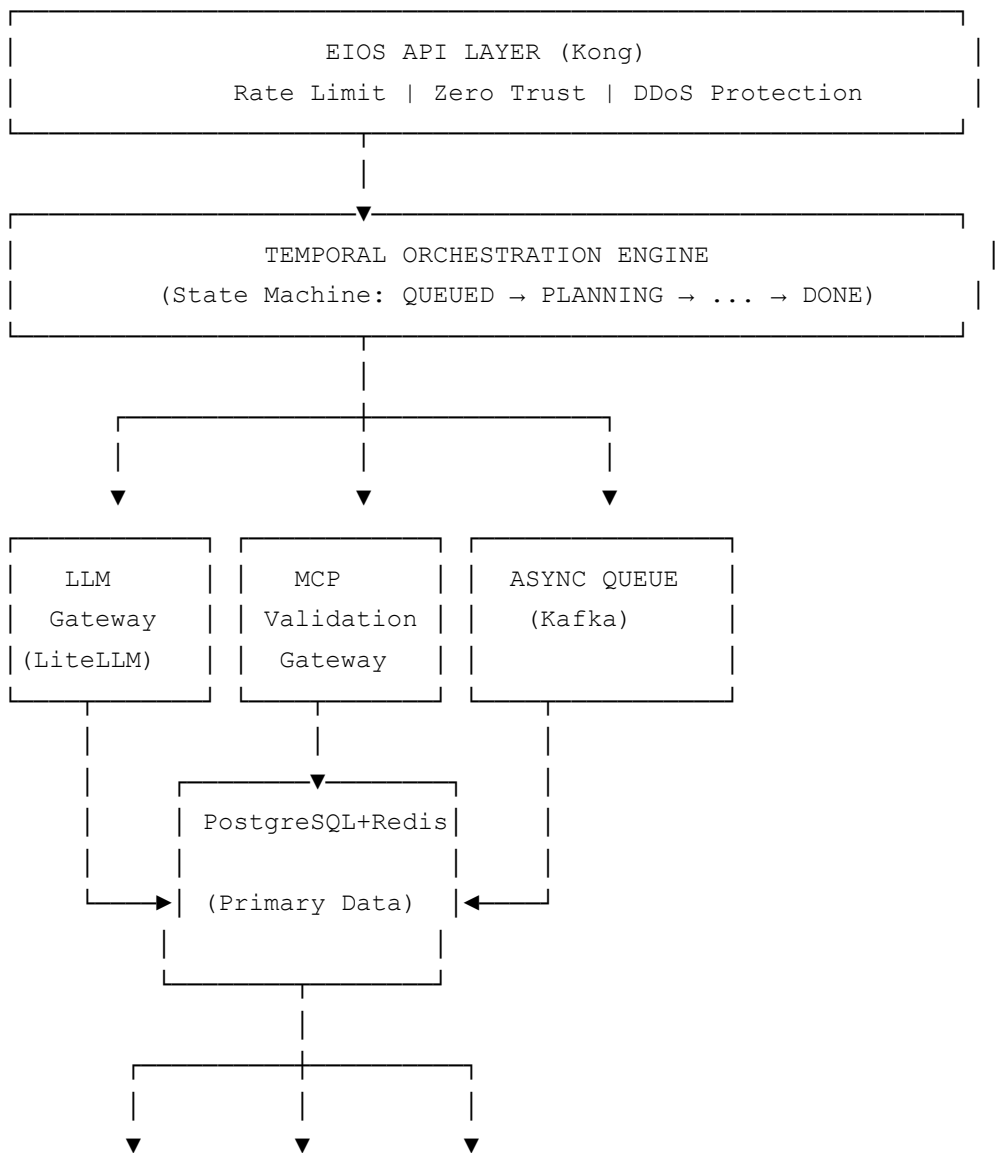
    // 4. Audit log
    await auditLog.record({
      agent_id: agentConfig.id,
```

```
    tool: toolName,  
    timestamp: now(),  
    approved: true  
  });  
  
  return true;  
}  
}
```

## 2 KIẾN TRÚC TỐI ƯU HÓA (RECOMMENDED)

### Phương pháp: “Incremental Integration”

Giữ Ollama Fleet của bạn, nhưng bao quanh nó với các thành phần Eios cần thiết:



|        |             |                    |
|--------|-------------|--------------------|
| YOUR   | Qdrant      | MCP Servers        |
| Ollama | (Vector DB) | (Postgres, Notion) |
| Fleet  |             |                    |

## Phase 1: MVP (Tuần 1-2)

### Mục tiêu: Ghép Ollama Fleet vào Eios core

#### Week 1: Setup Temporal + PostgreSQL

```
# Install Temporal Server (Docker)
docker run -d \
  --name temporal \
  -p 7233:7233 \
  temporalproto/server:1.24.0

# PostgreSQL
docker run -d \
  --name postgres \
  -e POSTGRES_PASSWORD=password \
  -p 5432:5432 \
  postgres:15

# Migrate Eios schema
psql -U postgres postgres < eios-schema.sql
```

#### Week 2: Implement LLM Gateway

```
// llm-gateway.ts
import Anthropic from "@anthropic-ai/sdk";
import { fetch } from "undici";

class EiosLLMGateway {
  private ollama_router: string = "http://localhost:11434";
  private anthropic: Anthropic = new Anthropic();

  async call(
    agentConfig: AgentConfig,
    messages: Array<{ role: string; content: string }>
  ) {
    const model = agentConfig.model; // "anthropic/claude-3-5-sonnet" or "ollama/"
```



```

if (model.startsWith("anthropic/")) {
  // Route to Anthropic API
  return await this.anthropic.messages.create({
    model: model.replace("anthropic/", ""),
    max_tokens: agentConfig.max_tokens,
    system: agentConfig.system_prompt,
    messages: messages,
  });
} else if (model.startsWith("ollama/")) {
  // Route to YOUR Ollama Fleet Router
  const modelName = model.replace("ollama/", "");
  const response = await fetch(`${this.ollama_router}/api/generate`, {
    method: "POST",
    body: JSON.stringify({
      model: modelName,
      prompt: messages[messages.length - 1].content,
      stream: false,
    }),
  });
  return await response.json();
}
}
}

```

---

## Phase 2: Scaling (Tuần 3-6)

### Mục tiêu: Multi-agent coordination + MCP validation

#### Setup Kafka

```

# docker-compose.kafka.yml
services:
  kafka:
    image: confluentinc/cp-kafka:7.5.0
    environment:
      KAFKA_BROKER_ID: 1
      KAFKA_ZOOKEEPER_CONNECT: zookeeper:2181
      KAFKA_ADVERTISED_LISTENERS: PLAINTEXT://kafka:29092,PLAINTEXT_HOST://localhost
      KAFKA_OFFSETS_TOPIC_REPLICATION_FACTOR: 1
    depends_on:
      - zookeeper

# Topics
topics:

```

- agent-execution-started
- agent-execution-step-completed
- review-requested
- subagent-task-assigned

## Implement Subagent Orchestration

```
// subagent-orchestrator.ts
class SubagentOrchestrator {
  async callSubagent(
    parentExecution: Execution,
    subagentId: string,
    task: string
  ) {
    // 1. Create child execution
    const childExecution = await db.executions.create({
      agent_id: subagentId,
      parent_execution_id: parentExecution.id,
      status: "QUEUED",
      input_context: { task },
    });

    // 2. Publish event
    await kafka.send({
      topic: "agent-execution-started",
      messages: [
        {
          value: JSON.stringify({
            execution_id: childExecution.id,
            agent_id: subagentId,
            task,
          }),
        },
      ],
    });

    // 3. Subscribe for result (with timeout)
    return new Promise((resolve, reject) => {
      const timeout = setTimeout(
        () => reject(new TimeoutError("Subagent timeout")),
        30 * 60 * 1000
      ); // 30 min timeout

      kafka.consumer.subscribe({
        topic: "agent-execution-step-completed",
```

```

    onMessage: (msg) => {
      const data = JSON.parse(msg.value);
      if (data.execution_id === childExecution.id) {
        clearTimeout(timeout);
        resolve(data.result);
      }
    },
  });
});
}
}

```

---

## Phase 3: Enterprise (Tuần 7-10)

### Mục tiêu: Vector DB + Advanced Security + Billing

#### Setup Qdrant

```

docker run -d \
  --name qdrant \
  -p 6333:6333 \
  qdrant/qdrant:v1.7.0

```

#### Implement RAG + Long-term Memory

```

// long-term-memory.ts
import OpenAI from "openai";
import Qdrant from "@qdrant/js-client-rest";

class LongTermMemory {
  private qdrant = new Qdrant({
    url: "http://localhost:6333",
  });
  private embeddings = new OpenAI();

  async storeExecution(execution: Execution) {
    // 1. Embed execution summary
    const embedding = await this.embeddings.embeddings.create({
      input: execution.summary,
      model: "text-embedding-3-small",
    });

    // 2. Store in Qdrant

```

```

await this.qdrant.upsert("agent-memory", {
  points: [
    {
      id: execution.id,
      vector: embedding.data[0].embedding,
      payload: {
        agent_id: execution.agent_id,
        summary: execution.summary,
        tokens_used: execution.tokens_used,
        timestamp: execution.completed_at,
      },
    },
  ],
});
}

async retrieveSimilarExecutions(
  agentId: string,
  query: string,
  limit: number = 3
) {
  // 1. Embed query
  const embedding = await this.embeddings.embeddings.create({
    input: query,
    model: "text-embedding-3-small",
  });

  // 2. Search Qdrant
  const results = await this.qdrant.search("agent-memory", {
    vector: embedding.data[0].embedding,
    limit,
    filter: {
      must: [
        {
          key: "agent_id",
          match: { value: agentId },
        },
      ],
    },
  });

  return results.result;
}
}

```

---

### 3 ROADMAP IMPLEMENTATION (Chi tiết)

| Tuần | Component                        | Priority                                                                                   | Effort | Output                       |
|------|----------------------------------|--------------------------------------------------------------------------------------------|--------|------------------------------|
| 1    | Temporal Setup + Schema          |  High     | 3 days | Local dev environment        |
| 2    | LLM Gateway (Anthropic + Ollama) |  High     | 3 days | First agent execution        |
| 3    | API Gateway (Kong)               |  Medium   | 2 days | Rate limiting + JWT          |
| 4    | PostgreSQL + Redis               |  High     | 2 days | Data persistence             |
| 5    | MCP Validation Layer             |  High     | 3 days | Secure external integrations |
| 6    | Kafka + Subagent Orchestration   |  Medium   | 4 days | Multi-agent support          |
| 7    | Qdrant + RAG                     |  Medium | 3 days | Long-term memory             |
| 8    | Web Dashboard (Next.js)          |  Medium | 5 days | Real-time monitoring         |
| 9    | CLI improvements                 |  Low    | 3 days | UX polish                    |
| 10   | Load testing + optimization      |  High   | 5 days | 99.9% SLA ready              |

### 4 QUICK START CHECKLIST

Phase 1 - MVP (Ready in 2 weeks):

- [x] Clone Eios repo
- [ ] Setup docker-compose with Temporal, PostgreSQL, Redis
- [ ] Implement LLM Gateway (route to Ollama fleet OR Anthropic)
- [ ] Implement State Machine executor (QUEUED → PLANNING → EXECUTING)
- [ ] CLI: `eios run "task"` → creates execution → polls status
- [ ] Test: Manual execution via CLI

Phase 2 - Orchestration (Weeks 3-6):

- [ ] Setup Kafka
- [ ] Implement Subagent calling (parent → child)
- [ ] Implement MCP Validation Layer
- [ ] Self-hosted agent pool management

Phase 3 - Enterprise (Weeks 7-10):

- [ ] Setup Qdrant + RAG
- [ ] Web Dashboard with Split-pane Review UI
- [ ] SSO/SAML integration
- [ ] Billing engine (per-token, per-execution)
- [ ] Carbon Impact tracking

---

## 5 HOW YOUR OLLAMA FLEET FITS IN

**Bạn không cần thay đổi gì trong ollama-fleet-router!**

Just wire it as one LLM provider:

```
// llm-gateway.ts
const PROVIDERS = {
  "anthropic/claude-3-5-sonnet": {
    type: "remote",
    endpoint: "https://api.anthropic.com/v1",
    key: process.env.ANTHROPIC_API_KEY,
  },
  "openai/gpt-4": {
    type: "remote",
    endpoint: "https://api.openai.com/v1",
    key: process.env.OPENAI_API_KEY,
  },
  "ollama/*": {
    type: "local",
    endpoint: "http://localhost:11434", // YOUR ROUTER
    key: null,
  },
};
```

**Lợi ích:**

- Agents có thể fallback từ OpenAI → Ollama nếu quota hết
- Cheaper inference cost dùng local Ollama

- Latency thấp (local models)
- Full control over models (security, compliance)



## COMPARISON TABLE

| Khía cạnh        | Ollama Fleet<br>(Hiện tại) | Full Eios Stack            | Khuyến cáo                |
|------------------|----------------------------|----------------------------|---------------------------|
| Scalability      | 1-10 agents                | 1000+ agents               | Use Temporal+PostgreSQL   |
| State Management | Simple SQLite              | Temporal durable workflows | CRITICAL for human review |
| Multi-agent      | ✗ No                       | ✓ Kafka coordination       | Need message broker       |
| Long-term Memory | ✗ No                       | ✓ Vector DB (Qdrant)       | Phase 3 enhancement       |
| Rate Limiting    | ✗ No                       | ✓ Kong API Gateway         | Essential for multi-team  |
| MCP Security     | ✗ Simple execution         | ✓ Validation layer         | CRITICAL for enterprise   |
| Observability    | ✓ Basic monitoring         | ✓✓ Comprehensive           | Upgrade dashboard         |
| Cost             | 📉 Very low                 | 📈 Moderate                 | Depends on scale          |

**Kết luận:** Bạn đang trên đúng hướng với Ollama Fleet! Bây giờ thêm Temporal + PostgreSQL để biến nó thành một nền tảng enterprise-ready cho Eios.